University of Computer Studies, Yangon (UCSY)

Mining Frequent Patterns, Associations, and Correlations: Basic Concepts and Methods

IS-212
(Data and Knowledge Mining)

Dr. Hsu Myat Mo

Faculty of Information Science, UCSY

References

- Data Mining Concepts and Techniques, Jiawei Han, Micheline Kamber, Jian Pei. 3rd Edition, Morgan Kaufmann.
- Introduction to Data Mining, Pang-Ning Tan, Michael Steinbach, Vipin Kumar, Addison Wesley, 2006.
- Data Mining – Practical Machine Learning Tools and Techniques (2nd Ed.), Ian H. Witten and Eibe Frank, Morgan Kaufmann, 2005.
- Predictive Data Mining, S.M. Weiss and N. Indurkhya, Morgan Kaufmann, 1998.
- Data Mining – Introductory and Advanced Topics, Margaret H. Dunham, Prentice Hall, 2003.
- Data Mining with R – Learning with Case Studies, Torgo, Luís, CRC Press, Taylor and Francis Group, LLC, 2011

Course Objectives

- To explain the workflow of data mining techniques
- To remember how to manipulate data mining techniques with Python and R-Programming
- To explain how to evaluate the experimental results of data mining techniques
- To express the visualization of the experimental results with Python and R-Programming

Outlines

- ➢Basic Concepts
- ➢Frequent Itemset Mining Methods
- ➢Which Patterns are Interesting? – Pattern Evaluation Methods
- ➢Summary

Introduction

- Association rule mining can be viewed as a two-stepprocess:

- 1. Find all frequent itemsets : By definition, each of these itemsets will occur at least as frequently as a predetermined minimum support count , minsup.
- 2.Generate strong association rules from the frequent itemsets : By definition, these rules must satisfy minimum support and minimum confidence

Data Mining Function: Association and Correlation Analysis

- Frequent patterns, are patterns that occur frequently in data.
  - frequent itemsets, frequent subsequences (also known as sequential patterns), and frequent substructures.
  - A frequent itemset typically refers to a set of items that often appear together in a transactional data set
- Frequent patterns (or frequent itemsets)
  - What items are frequently purchased together in your Walmart?
- Association, correlation vs. causality
  - A typical association rule
    - Diaper → Beer [0.5%, 75%] (support, confidence)
  - Are strongly associated items also strongly correlated?
- How to mine such patterns and rules efficiently in large datasets?
- How to use such patterns for classification, clustering, and other applications?

Closed Patterns and Max-Patterns

- A long pattern contains a combinatorial number of sub-patterns, e.g., {a1, …, a100} contains $\binom{100}{1} + \binom{100}{2} + \cdots + \binom{100}{100} = 2^{100} - 1 \approx 1.27 \times 10^{30}$ sub-patterns!
- Solution: Mine closed frequent itemsets and maximal frequent itemsets instead
- A pattern *p* is a closed pattern if there is no superpattern *p'* with the same support as p.
  - An itemset X is closed in a data set D if there exists no proper super-itemset Y such that Y has the same support count as X in D.
  - An itemset X is a closed frequent itemset in set D if X is both closed and frequent in D.
- Pattern *p* is a max-pattern if there exists no frequent superpattern of *p*.
  - An itemset X is a maximal frequent itemset (or max-itemset) in a data set D if X is frequent, and there exists no super-itemset Y such that X Y and Y is frequent in D.
- Closed pattern is a lossless compression of freq. patterns

Closed Patterns and Max-Patterns

- Exercise: Suppose a DB contains only two transactions
  - <a_1, …, a_{100}>, <a_1, …, a_{50}>
  - Let min_sup = 1
- What is the set of closed frequent itemsets?
  - {a_1, …, a_{100}}: 1
  - {a_1, …, a_{50}}: 2
- What is the set of maximal frequent itemsets?
  - {a_1, …, a_{100}}: 1
- What is the set of all itemsets?
  - {a_1}: 2, …, {a_1, a_2}: 2, …, {a_1, a_{51}}: 1, …, {a_1, a_2, …, a_{100}}: 1
  - A big number: 2^{100} - 1

Basic Concepts: k-Itemsets and Their Supports

- Itemset: A set of one or more items

$$\mathrm{I} = \{ I_1, I_2, \cdots, I_m \}$$

- k-itemset: An itemset containing k items:

  $X = \{x_1, \ldots, x_k\}$

  - Ex. {Beer, Nuts, Diaper} is a 3-itemset

- Absolute support (count)
  - sup{X} = occurrences of an itemset X
    - Ex. sup{Beer} = 3
    - Ex. sup{Diaper} = 4
    - Ex. sup{Beer, Diaper} = 3
    - Ex. sup{Beer, Eggs} = 1

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

- Relative support
  - *s*{*X*} = The fraction of transactions that contains X (i.e., the probability that a transaction contains X)
  - Ex. s{Beer} = 3/5 = 60%
  - Ex. s{Diaper} = 4/5 = 80%
  - Ex. s{Beer, Eggs} = 1/5 = 20%

Frequent Itemset Mining Methods

- Apriori: A Candidate Generation-and-Test Approach
  - Improving the Efficiency of Apriori
- FPGrowth: A Frequent Pattern-Growth Approach
- Frequent Pattern Mining with Vertical Data Format

Apriori Algorithm

- Proposed by R. Agrawal and R. Srikanth in 1994
- Name of the algorithm is based on the fact that the algorithm uses prior knowledge of the frequent itemset properties
- Apriori employs an iterative approach known as a level-wise search where k-itemsets are used to explore (k+1)-itemsets

- Apriori property**:** *All nonempty subsets of a frequent itemset must also be frequent*

- *How is the Apriori property used in the algorithm?"*
- *Method:* A two-step process is followed, consisting of **join** and **prune** actions.

Apriori Method

- Method:
  - Initially, scan DB once to get frequent 1-itemset
  - Generate length (k+1) candidate itemsets from length k frequent itemsets
  - Terminate when no frequent or candidate set can be generated
- To improve the efficiency of the level-wise generation of frequent itemsets, the apriori property is used to reduce the search space
  - All non-empty subsets of a frequent itemset must also be frequent
  - If {beer, diaper, nuts} is frequent, so is {beer, diaper}

Implementation of Apriori

- How to generate candidates?
  - Step 1: self-joining L_k
  - Step 2: pruning
- Example of Candidate-generation
  - L_3={*abc, abd, acd, ace, bcd*}
  - Self-joining: L_3*L_3
    - *abcd* from *abc* and *abd*
    - *acde* from *acd* and *ace*
  - Pruning:
    - *acde* is removed because *ade* is not in L_3
  - C_4 = {*abcd*}

Algorithm: Apriori. Find frequent itemsets using an iterative level-wise approach based on candidate generation.

Input:

- D, a database of transactions;
- min_sup, the minimum support count threshold.

Output: L, frequent itemsets in D.

Method:

```
L_1 = find_frequent_1-itemsets(D);
for (k = 2; L_{k-1} ≠ φ; k++) {
    C_k = apriori_gen(L_{k-1});
    for each transaction t ∈ D { // scan D for counts
        C_t = subset(C_k, t); // get the subsets of t that are candidates
        for each candidate c ∈ C_t
            c.count++;
    }
    L_k = {c ∈ C_k | c.count ≥ min_sup}
}
return L = ∪_k L_k;

procedure apriori_gen(L_{k-1}:frequent (k − 1)-itemsets)
    for each itemset l_1 ∈ L_{k-1}
        for each itemset l_2 ∈ L_{k-1}
            if (l_1[1] = l_2[1]) ∧ (l_1[2] = l_2[2])
                ∧... ∧ (l_1[k − 2] = l_2[k − 2]) ∧ (l_1[k − 1] < l_2[k − 1]) then {
                c = l_1 ⋈ l_2; // join step: generate candidates
                if has_infrequent_subset(c, L_{k-1}) then
                    delete c; // prune step: remove unfruitful candidate
                else add c to C_k;
            }
    return C_k;

procedure has_infrequent_subset(c: candidate k-itemset;
            L_{k-1}: frequent (k − 1)-itemsets); // use prior knowledge
    for each (k − 1)-subset s of c
        if s ∉ L_{k-1} then
            return TRUE;
    return FALSE;
```

The Apriori Algorithm—An Example

$Sup_{min} = 2$

Database TDB

Tid	Items
10	A, C, D
20	B, C, E
30	A, B, C, E
40	B, E

1st scan

C_1

Itemset	sup
{A}	2
{B}	3
{C}	3
{D}	1
{E}	3

L_1

Itemset	sup
{A}	2
{B}	3
{C}	3
{E}	3

C_2

Itemset
{A, B}
{A, C}
{A, E}
{B, C}
{B, E}
{C, E}

2nd scan

C_2

Itemset	sup
{A, B}	1
{A, C}	2
{A, E}	1
{B, C}	2
{B, E}	3
{C, E}	2

L_2

Itemset	sup
{A, C}	2
{B, C}	2
{B, E}	3
{C, E}	2

C_3

Itemset
{B, C, E}

3rd scan

L_3

Itemset	sup
{B, C, E}	2

The Apriori Algorithm—An Example

Transactional Data for an *AllElectronics* Branch

TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Scan D for count of each candidate →

C_1

Itemset	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Compare candidate support count with minimum support count →

L_1

Itemset	Sup. count
{I1}	6
{I2}	7
{I3}	6
{I4}	2
{I5}	2

Generate C_2 candidates from L_1 →

C_2

Itemset
{I1, I2}
{I1, I3}
{I1, I4}
{I1, I5}
{I2, I3}
{I2, I4}
{I2, I5}
{I3, I4}
{I3, I5}
{I4, I5}

Scan D for count of each candidate →

C_2

Itemset	Sup. count
{I1, I2}	4
{I1, I3}	4
{I1, I4}	1
{I1, I5}	2
{I2, I3}	4
{I2, I4}	2
{I2, I5}	2
{I3, I4}	0
{I3, I5}	1
{I4, I5}	0

Compare candidate support count with minimum support count →

L_2

Itemset	Sup. count
{I1, I2}	4
{I1, I3}	4
{I1, I5}	2
{I2, I3}	4
{I2, I4}	2
{I2, I5}	2

Generate C_3 candidates from L_2 →

C_3

Itemset
{I1, I2, I3}
{I1, I2, I5}

Scan D for count of each candidate →

C_3

Itemset	Sup. count
{I1, I2, I3}	2
{I1, I2, I5}	2

Compare candidate support count with minimum support count →

L_3

Itemset	Sup. count
{I1, I2, I3}	2
{I1, I2, I5}	2

Fig: Generation of the candidate itemsets and frequent itemsets, where the minimum support count is 2

Improving the Efficiency of Apriori

- Hash-based technique
  - Hashing itemsets into corresponding buckets
  - A hash-based technique can be used to reduce the size of the candidate k-itemsets, C_k, for k > 1.
  - For example, when scanning each transaction in the database to generate the frequent 1-itemsets, L1, we can generate all the 2-itemsets for each transaction, hash (i.e., map) them into the different buckets of a hash table structure, and increase the corresponding bucket counts
  - A 2-itemset with a corresponding bucket count in the hash table that is below the support threshold cannot be frequent and thus should be removed from the candidate set.
  - Such a hash-based technique may substantially reduce the number of candidate k-itemsets examined (especially when k=2).

Cont'd

Create hash table H_2 using hash function $h(x, y) = ((order\ of\ x) \times 10 + (order\ of\ y))\ mod\ 7$

H_2

bucket address	0	1	2	3	4	5	6
bucket count	2	2	4	2	2	4	4
bucket contents	{I1, I4} {I3, I5}	{I1, I5} {I1, I5}	{I2, I3} {I2, I3} {I2, I3} {I2, I3}	{I2, I4} {I2, I4}	{I2, I5} {I2, I5}	{I1, I2} {I1, I2} {I1, I2} {I1, I2}	{I1, I3} {I1, I3} {I1, I3} {I1, I3}

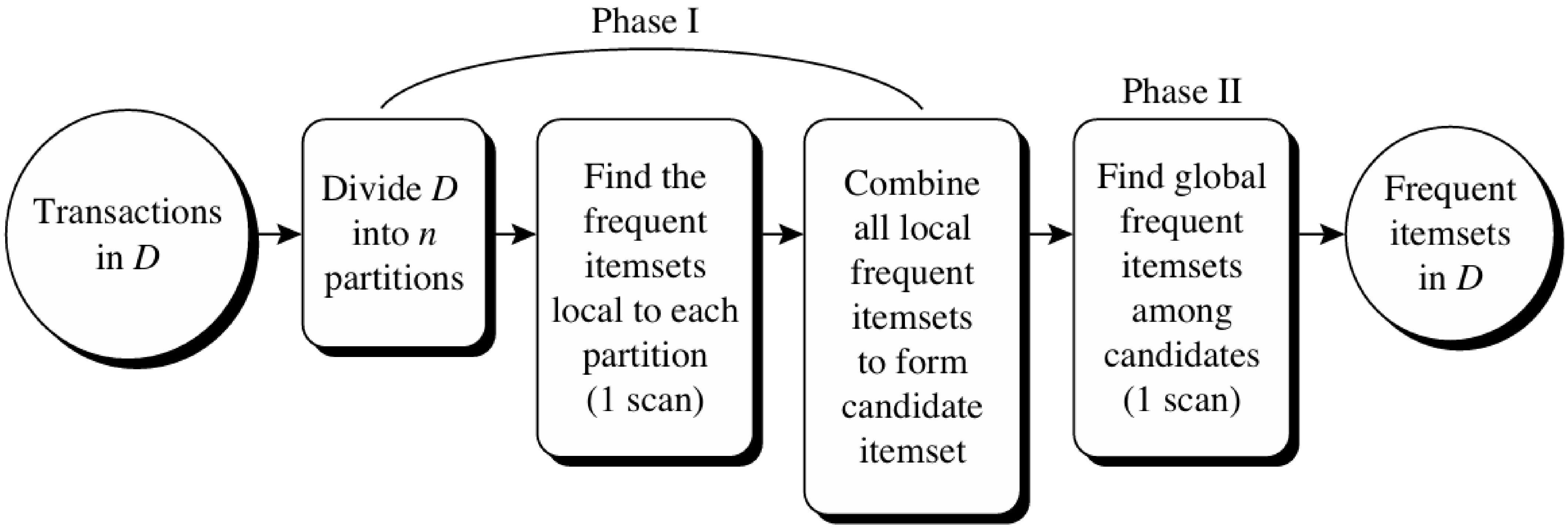
Hash table, H2, for candidate 2-itemsets.

Cont’d

- Transaction reduction
    - Reducing the number of transactions scanned in future iterations
    - A transaction that does not contain any frequent k-itemsets cannot contain any frequent (k +1)-itemsets.
    - Therefore, such a transaction can be marked or removed from further consideration because subsequent database scans for j-itemsets, where j >k, will not need to consider such a transaction.

Cont'd

- Partitioning
  - partitioning the data to find candidate itemsets



Cont'd

- Sampling
  - Mining on a subset of the given data
  - The basic idea of the sampling approach is to pick a random sample S of the given data D, and then search for frequent itemsets in S instead of D. In this way, we trade off some degree of accuracy against efficiency.
  - Because we are searching for frequent itemsets in S rather than in D, it is possible that we will miss some of the global frequent itemsets.
  - The sampling approach is especially beneficial when efficiency is of utmost importance such as in computationally intensive applications that must be run frequently.

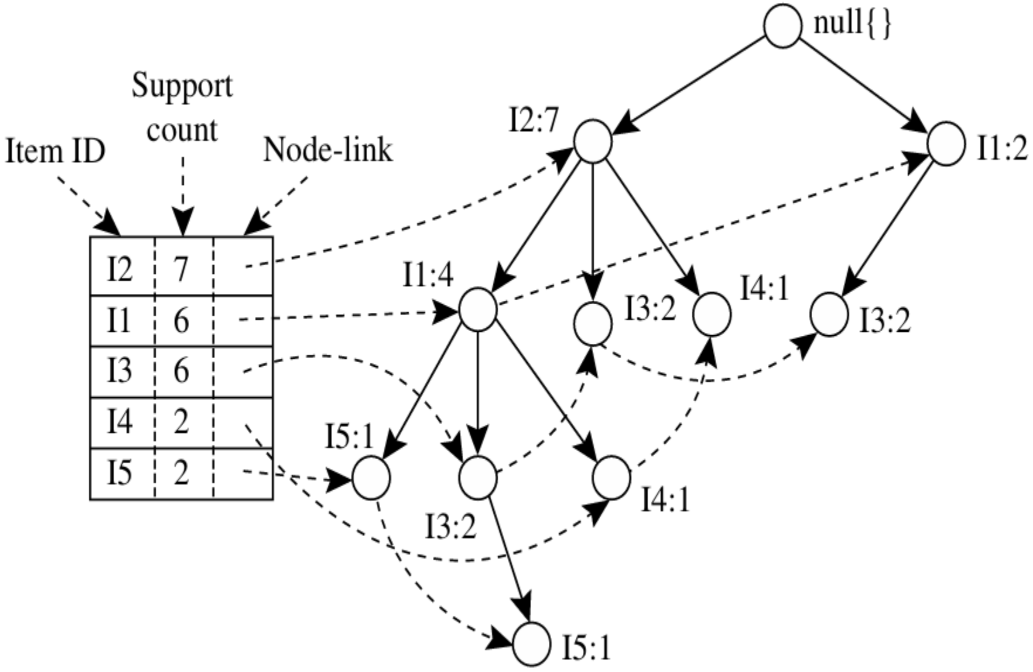
Cont'd

- Dynamic itemset counting
  - Adding candidate itemsets at different points during a scan
  - New candidate itemsets can be added at any start point, unlike in Apriori, which determines new candidate itemsets only immediately before each complete database scan.

A Pattern-Growth Approach for Mining Frequent Itemsets

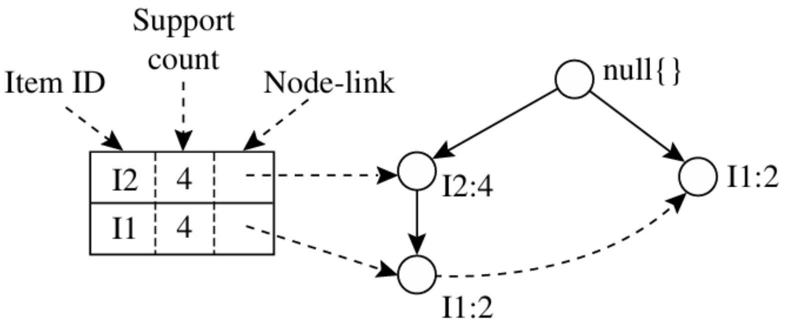
- Frequent pattern growth or FP-growth
- Adopts a divide-and-conquer strategy
- It compresses the database representing frequent items into a frequent pattern tree, or FP-tree, which retains the itemset association information.
- It then divides the compressed database into a set of conditional databases (a special kind of projected database), each associated with one frequent item or “pattern fragment,” and mines each database separately.
- The FP-growth method transforms the problem of finding long frequent patterns into searching for shorter ones in much smaller conditional databases recursively and then concatenating the suffix.
- It uses the least frequent items as a suffix, offering good selectivity.
- The method substantially reduces the search costs.

Cont’d



Mining the FP-Tree by Creating Conditional (Sub-)Pattern Bases

Item	*Conditional Pattern Base*	*Conditional FP-tree*	*Frequent Patterns Generated*
I5	{{I2, I1: 1}, {I2, I1, I3: 1}}	⟨I2: 2, I1: 2⟩	{I2, I5: 2}, {I1, I5: 2}, {I2, I1, I5: 2}
I4	{{I2, I1: 1}, {I2: 1}}	⟨I2: 2⟩	{I2, I4: 2}
I3	{{I2, I1: 2}, {I2: 2}, {I1: 2}}	⟨I2: 4, I1: 2⟩, ⟨I1: 2⟩	{I2, I3: 4}, {I1, I3: 4}, {I2, I1, I3: 2}
I1	{{I2: 4}}	⟨I2: 4⟩	{I2, I1: 4}



Mining by Exploring Vertical Data Format

Minimum support = 2

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

itemset	TID_set
I1	{T100, T400, T500, T700, T800, T900}
I2	{T100, T200, T300, T400, T600, T800, T900}
I3	{T300, T500, T600, T700, T800, T900}
I4	{T200, T400}
I5	{T100, T800}

itemset	TID_set
{I1, I2}	{T100, T400, T800, T900}
{I1, I3}	{T500, T700, T800, T900}
{I1, I4}	{T400}
{I1, I5}	{T100, T800}
{I2, I3}	{T300, T600, T800, T900}
{I2, I4}	{T200, T400}
{I2, I5}	{T100, T800}
{I3, I5}	{T800}

itemset	TID_set
{I1, I2, I3}	{T800, T900}
{I1, I2, I5}	{T100, T800}

Association Rules

- Retail shops are often interested in associations between different items that people buy.
  - Someone who buys bread is quite likely also to buy milk
  - A person who bought the book *Database System Concepts* is quite likely also to buy the book *Operating System Concepts*.
- Associations information can be used in several ways.
  - E.g. when a customer buys a particular book, an online shop may suggest associated books.
- **Association rules:**
- *bread* $\Rightarrow$ *milk*   *DB-Concepts, OS-Concepts* $\Rightarrow$ Networks
  - Left hand side: **antecedent**,   righthand side: **consequent**
  - An association rule must have an associated **population**; the population consists of a set of **instances**
    - E.g. each transaction (sale) at a shop is an instance, and the set of all transactions is the population

Association Rules

- Retail shops are often interested in associations between different items that people buy.
  - Someone who buys bread is quite likely also to buy milk
  - A person who bought the book *Database System Concepts* is quite likely also to buy the book *Operating System Concepts*.
- Associations information can be used in several ways.
  - E.g. when a customer buys a particular book, an online shop may suggest associated books.
- **Association rules:**
- $bread \Rightarrow milk$ $DB\text{-}Concepts, OS\text{-}Concepts \Rightarrow$ Networks
  - Left hand side: **antecedent**, righthand side: **consequent**
  - An association rule must have an associated **population**; the population consists of a set of **instances**
    - E.g. each transaction (sale) at a shop is an instance, and the set of all transactions is the population

Association Rules

- Rules have an associated support, as well as an associated confidence.
- **Support** is a measure of what fraction of the population satisfies both the antecedent and the consequent of the rule.
  - E.g., suppose only 0.001 percent of all purchases include milk and screwdrivers. The support for the rule is *milk* $\Rightarrow$ *screwdrivers* is low.
- **Confidence** is a measure of how often the consequent is true when the antecedent is true.
  - E.g., the rule *bread* $\Rightarrow$ *milk* has a confidence of 80 percent if 80 percent of the purchases that include bread also include milk.

$$buys(X, \text{"computer"}) \Rightarrow buys(X, \text{"software"}) \; [support = 1\%, confidence = 50\%],$$

$$age(X, \text{"20..29"}) \wedge income(X, \text{"40K..49K"}) \Rightarrow buys(X, \text{"laptop"})$$
$$[support = 2\%, confidence = 60\%].$$

Cont'd

$$support(A \Rightarrow B) = P(A \cup B)$$

$$confidence(A \Rightarrow B) = P(B|A).$$

$$confidence(A \Rightarrow B) = P(B|A) = \frac{support(A \cup B)}{support(A)} = \frac{support\_count(A \cup B)}{support\_count(A)}.$$

Association Rules

Relative support

$s\{X\}$ = The fraction of transactions that contains X (i.e., the probability that a transaction contains X)

Support

Ex. s{Diaper, Beer} = 3/5 = 0.6 (i.e., 60%)

Confidence of

The *conditional probability* that a transaction containing X also contains *Y:*

$c = \text{sup}(X, Y) / \text{sup}(X)$

Ex. *c* = sup{Diaper, Beer}/sup{Diaper} = ¾ = 0.75

Tid	Items bought
1	Beer, Nuts, Diaper
2	Beer, Coffee, Diaper
3	Beer, Diaper, Eggs
4	Nuts, Eggs, Milk
5	Nuts, Coffee, Diaper, Eggs, Milk

Generating Association Rules Example

- Example: Given the following table and the frequent itemset X = {I1. I2. I5}. generate the association rules.

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

- The nonempty subsets of X are:
- {I1, I2}, {I1, I5}, {I2, I5}, {I1}, {I2}, {I5}
- The resulting association rules are:

$$
\begin{aligned}
&\{I1, I2\} \Rightarrow I5, && confidence = 2/4 = 50\% \\
&\{I1, I5\} \Rightarrow I2, && confidence = 2/2 = 100\% \\
&\{I2, I5\} \Rightarrow I1, && confidence = 2/2 = 100\% \\
&I1 \Rightarrow \{I2, I5\}, && confidence = 2/6 = 33\% \\
&I2 \Rightarrow \{I1, I5\}, && confidence = 2/7 = 29\% \\
&I5 \Rightarrow \{I1, I2\}, && confidence = 2/2 = 100\%
\end{aligned}
$$

- If the minimum confidence threshold is, say, 70%, then only the second, third, and last rules are output, because these are the only ones generated that are strong

A misleading "strong" association rule

- Suppose we are interested in analyzing transactions at *AllElectronics* with respect to the purchase of computer games and videos. Let *game* refer to the transactions containing computer games, and *video* refer to those containing videos. Of the 10,000 transactions analyzed, the data show that 6000 of the customer transactions included computer games, while 7500 included videos, and 4000 included both computer games and videos. Suppose that a data mining program for discovering association rules is run on the data, using a minimum support of, say, 30% and a minimum confidence of 60%. The following association rule is discovered:

$$buys(X, \text{"computer games"}) \Rightarrow buys(X, \text{"videos"})$$

$$[support = 40\%, confidence = 66\%].$$

Rule is misleading because the probability of purchasing videos is 75%, which is even larger than 66%.

$$A \Rightarrow B\ [support, confidence, correlation]$$

Interestingness Measure: Correlations (Lift)

- *play basketball* $\Rightarrow$ *eat cereal* [40%, 66.7%]
- *play basketball* $\Rightarrow$ *not eat cereal* [20%, 33.3%]
- Measure of dependent/correlated events: lift $lift = \frac{P(A \cup B)}{P(A)P(B)}$
- If lift =1, occurrence of itemset A is independent of occurrence of itemset B, otherwise, they are dependent and correlated; if >1, positively correlated and <1, negatively correlated.

$$lift(B,C) = \frac{2000/5000}{3000/5000*3750/5000} = 0.89$$

$$lift(B,\neg C) = \frac{1000/5000}{3000/5000*1250/5000} = 1.33$$

	Basketball	Not basketball	Sum (row)
Cereal	2000	1750	3750
Not cereal	1000	250	1250
Sum(col.)	3000	2000	5000

Correlations (Lift)

- the probability of purchasing a computer game is *P({game})* = 0.60,
- the probability of purchasing a video is *P*({*video*})= 0.75, and
- the probability of purchasing both is *P*({*game*, *video*}) = 0.40.
- the lift of Rule is *P({game, video})/(P({game})*P({video}))* = 0.40/(0.60*0.75)= 0.89.

2 × 2 Contingency Table Summarizing the Transactions with Respect to Game and Video Purchases

	$game$	$\overline{game}$	Σ_{row}
$video$	4000	3500	7500
$\overline{video}$	2000	500	2500
Σ_{col}	6000	4000	10,000

- Because this value is less than 1,
- there is a negative correlation between the occurrence of {game} and {video}.

Correlation Analysis (Nominal Data)

- X^2 (chi-square) test

$$\chi^2 = \sum \frac{(Observed - Expected)^2}{Expected}$$

- The larger the X2 value, the more likely the variables are related
- The cells that contribute the most to the X^2 value are those whose actual count is very different from the expected count
- Correlation does not imply causality
- # of hospitals and # of car-theft in a city are correlated
- Both are causally linked to the third variable: population

Correlation analysis using X^2.

Contingency Table, with the Expected Values

	game	$\overline{game}$	Σ_{row}
video	4000 (4500)	3500 (3000)	7500
$\overline{video}$	2000 (1500)	500 (1000)	2500
Σ_{col}	6000	4000	10,000

$$\chi^2 = \Sigma \frac{(observed - expected)^2}{expected} = \frac{(4000-4500)^2}{4500} + \frac{(3500-3000)^2}{3000} + \frac{(2000-1500)^2}{1500} + \frac{(500-1000)^2}{1000} = 555.6.$$

A Comparison of Pattern Evaluation Measures

- Given two itemsets, *A* and *B*, the **all confidence** measure of *A* and *B* is defined as:

$$all\_conf(A,B) = \frac{sup(A \cup B)}{max\{sup(A), sup(B)\}} = min\{P(A|B), P(B|A)\}$$

- Given two itemsets, *A* and *B*, the **max confidence** measure of *A* and *B* is defined as:

$$max\_conf(A, B) = max\{P(A \mid B), P(B \mid A)\}$$

- Given two itemsets, *A* and *B*, the **Kulczynski** measure of *A* and *B* (**Kulc**) is defined as:

$$Kulc(A, B) = \frac{1}{2}(P(A|B) + P(B|A))$$

- Given two itemsets, *A* and *B*, the **cosine** measure of *A* and *B* is defined as :

$$cosine(A, B) = \frac{P(A \cup B)}{\sqrt{P(A) \times P(B)}} = \frac{sup(A \cup B)}{\sqrt{sup(A) \times sup(B)}}$$

$$= \sqrt{P(A|B) \times P(B|A)}.$$

Cont’d

2 × 2 Contingency Table for Two Items

	milk	$\overline{milk}$	Σ_{row}
coffee	mc	$\overline{m}c$	c
$\overline{coffee}$	$m\overline{c}$	$\overline{mc}$	$\overline{c}$
Σ_{col}	m	$\overline{m}$	Σ

Null invariance means: The number of null transactions does not matter. Does not change the measure value.

Comparison of Six Pattern Evaluation Measures Using Contingency Tables for a Variety of Data Sets

Data Set	mc	$\overline{m}c$	$m\overline{c}$	$\overline{mc}$	χ^2	*lift*	*all_conf.*	*max_conf.*	*Kulc.*	*cosine*
D_1	10,000	1000	1000	100,000	90557	9.26	0.91	0.91	0.91	0.91
D_2	10,000	1000	1000	100	0	1	0.91	0.91	0.91	0.91
D_3	100	1000	1000	100,000	670	8.44	0.09	0.09	0.09	0.09
D_4	1000	1000	1000	100,000	24740	25.75	0.5	0.5	0.5	0.5
D_5	1000	100	10,000	100,000	8173	9.18	0.09	0.91	0.5	0.29
D_6	1000	10	100,000	100,000	965	1.97	0.01	0.99	0.5	0.10

Cont'd

- "Among the all confidence, max confidence, Kulczynski, and cosine measures, which is best at indicating interesting pattern relationships?"
- To answer this question,
- Introduce the imbalance ratio (IR), which assesses the imbalance of two itemsets, A and B, in rule implications.

$$IR(A,B) = \frac{|sup(A) - sup(B)|}{sup(A) + sup(B) - sup(A \cup B)}$$

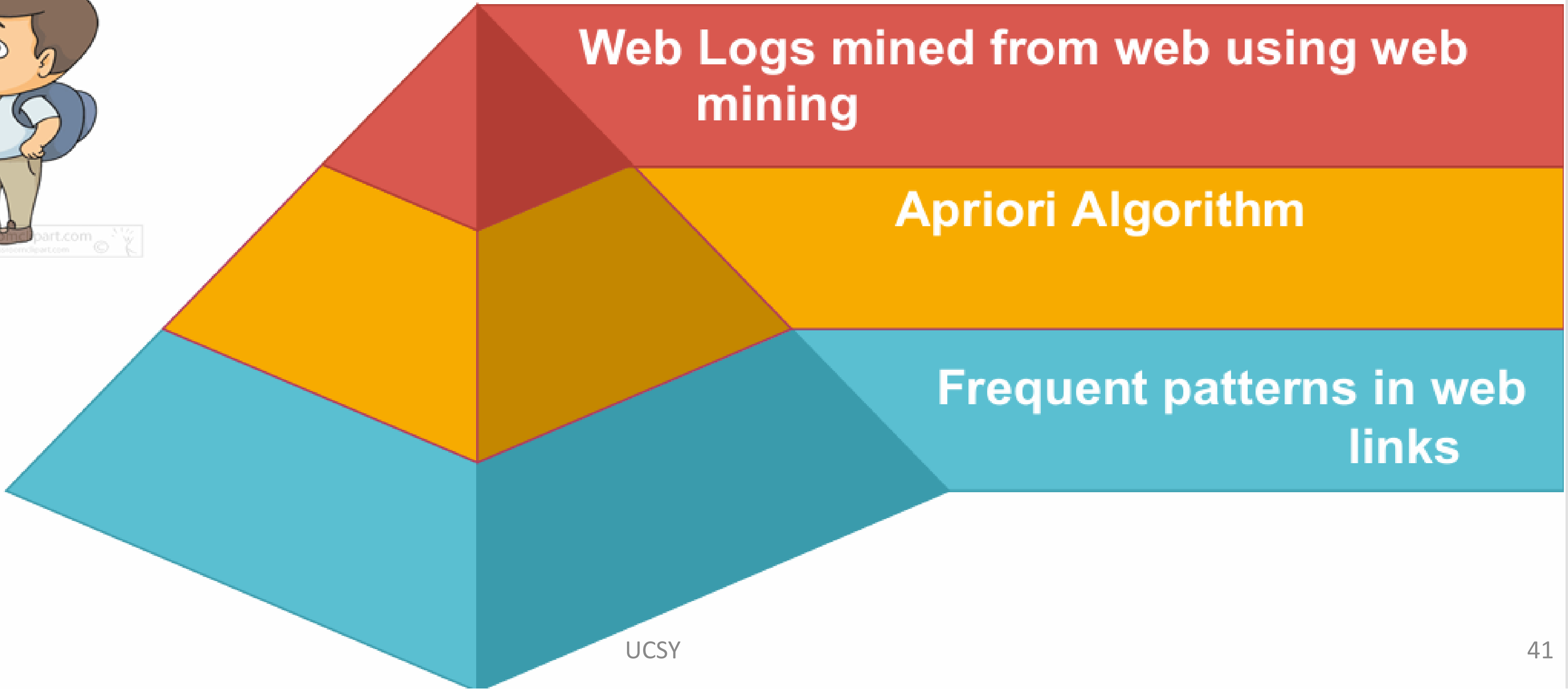
Summary of Research paper "Frequent Pattern Mining in Web Log Data using Apriori Algorithm"

Authors : S.VijayaKumar, A.S.Kumaresan, U.Jayalakshmi

International Journal of Emerging Engineering Research and Technology (IJEERT)

Volume 3, Issue 10, October 2015

Key Concepts Used



Step by step derivation of interesting rules using Apriori Algorithm on web logs

1

WEB LOG DATA

IP Address	URL Accessed	Time
rres1,ne.wi.ac.uk	/api/java.io.Bufferedwriter,html	01/Jan/1999
	/api/java.util.zip, CRc32.html	01/Jan/1999
	/api/java.io.Bufferedwriter,html	02/Feb/1999
	/java-tutorial/ui/animLoop.html	04/Feb/1999
	/atm /logiciels,html\	18/Feb/1999
	/relnotes/deprecatedlist.html	18/Feb/1999
Acasun.cekerd.edu	/perl/perlre.html	11/Jan/1999
	/java.tutorial/animLoop.html	12/Jan/1999
	/html4,0/struct/global,html	29/Jan/1999
	/api/java.util.zip,CRC32,html	29/Jan/1999
	/postgres/html-manual/query.html	29/Jan/1999
Accès.francomedia.gc.ca	/java.tutorial/animLoop.html	05/Jan/1999
	/apache/manual/misc/API.html	05/Jan/1999
	/postgres/html-manual/query.html	05/Jan/1999
	/perl/perlre.html	12/Feb/1999
	/api/java.io.Bufferedwriter,html	12/Feb/1999
ach3.pharma.mcgill.ca	api/java.io.Bufferedwriter,html	06/Feb/1999
	/java-tutorial/ui/animLoop.html	06/Feb/1999
	/html4,0/struct/global,html	07/Feb/1999
	/postgres/html-manual/query.html	07/Feb/1999
	/relnotes/deprecatedlist.html	08/Feb/1999

Minimum Support count = 2
Minimum confidence = 60%

http://www.ijeert.org/pdf/v3-i10/9.pdf

2 Numbering URLs

URL	No
/api/java.io.Bufferedwriter,html	1
/api/java.util.zip, CRc32.html	2
/java-tutorial/ui/animLoop.html	3
/atm /logiciels,html\	4
/relnotes/deprecatedlist.html	5
/perl/perlre.html	6
/html4,0/struct/global,html	7
/postgres/html-manual/query.html	8
/apache/manual/misc/API.html	9

3 Summary of Web log Data

Ip Address	URL Accessed
A	1,2,1,3,4,5
B	6,3,7,2,8
C	3,9,8,6,1
D	1,3,7,8,5

4 Scan database for count of each candidate

URL	Support Count
1	4
2	2
3	4
4	1
5	2
6	2
7	2
8	3
9	1

5 Calculation of L1

URL	Support Count
1	4
2	2
3	4
5	2
6	2
7	2
8	3

6 Calculation of L2

URL	Support Count
1,3	3
1,5	2
1,8	2
2,3	2
3,5	2
3,6	2
3.7	2
3,8	3
6,8	2
7,8	2

http://www.ijeert.org/pdf/v3-i10/9.pdf

7 Calculation of L3

URL	Support Count
1,3,5	2
1,3,8	2
3,6,8	2
3,2,8	2

8.1 Calculation of Confidence for L2

URL	Total Occurrences of X&Y	Total occurrences of X	Confidence
1,3	3	4	0.75
1,5	2	4	0.5
1,8	2	4	0.5
2,3	2	2	1
3,5	2	4	0.5
3,6	2	4	0.5
3,7	2	4	0.5
3,8	3	4	0.75
6,8	2	2	1
7,8	2	2	1

8.2 Calculation of Confidence for L3

URL	Total Occurrences of X&Y	Total occurrences of X	Confidence
<1,3>,5	2	3	0.67
<1,3>,8	2	3	0.67
<3,6>,8	2	2	1
<3,2>,8	2	2	1

http://www.ijeert.org/pdf/v3-i10/9.pdf

Conclusion of Research paper

- It introduced the process of web log mining, and to show how frequent pattern discovery tasks can be applied on the web log data in order to obtain useful information about the user's navigation behavior.

End of Lecture!

hsumyatmo@ucsy.edu.mm